

Triton kernel integration tutorial

HSiKAN inner-forward kernel

HyMeKo / HSiKAN team

2026-05-09

Abstract

Reference document for porting any other PyTorch hot-path in HSiKAN to a Triton kernel. Written 2026-05-09 after landing the SignedKAN inner-forward kernel: $60\times$ no-skip / $38\times$ highway speedup, 94.6% peak-memory savings vs the PyTorch path, parity $\leq 2 \cdot 10^{-7}$ against the production `SignedKANLayer.forward`. Live integration is env-gated (default off) until the fused backward kernel lands.

Contents

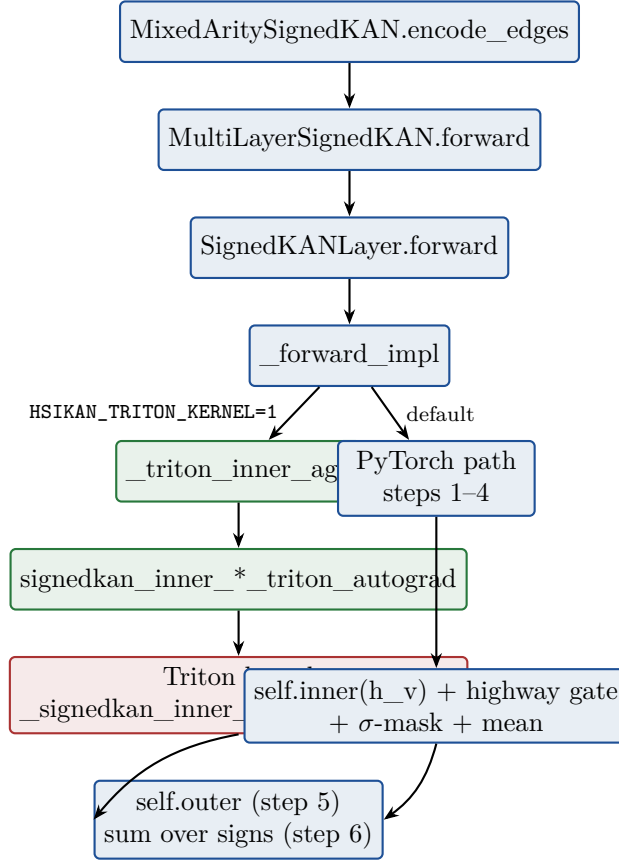
1	Why	1
2	Where the kernel lives in the call graph	2
3	Build process — six steps, in order	2
4	Test tiers	3
5	The math — forward and backward in closed form	4
5.1	Forward (Highway variant, matches production)	4
5.2	Backward — what the fused backward kernel must compute	5
5.2.1	(a) $\partial\mathcal{L}/\partial x$ — three contributions	5
5.2.2	(b) $\partial\mathcal{L}/\partial C^+, \partial\mathcal{L}/\partial C^-$ — control-point gradients	5
5.2.3	(c) $\partial\mathcal{L}/\partial W, \partial\mathcal{L}/\partial b$ — gate Linear gradients	6
5.3	Memory profile of the fused backward	6
6	Forward-semantics comparison: before vs after the fix	6
7	The OOM trap and why integration is env-gated	7
8	Common pitfalls — checklist	7
9	Files of record	7
10	Quick reference — running the suite	8

1 Why

The HSiKAN training loop spends most of its time in one path: gather per-vertex embeddings, apply a Catmull-Rom spline per sign, apply an optional Highway skip, mask by σ , mean-reduce. At Slashdot SOTA shape ($T = 200K$, $d = 4$, $k = 4$) this PyTorch path costs 11–17 ms per forward and allocates a 343 MB (T, k, S, d) intermediate. A fused Triton kernel collapses it to one launch, no intermediate, at 0.2–0.4 ms.

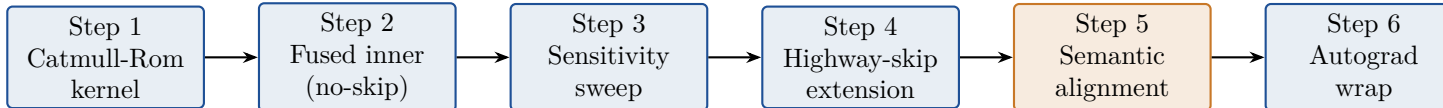
The kernel is the path that gates whether training stays under the VRAM cap on consumer GPUs and whether ablations finish in hours instead of days.

2 Where the kernel lives in the call graph



The kernel covers steps 1–4 of `_forward_impl` (gather, per-sign CR, highway-skip, σ -mask, mean reduce). Steps 5–6 (outer spline + sum-over-signs) stay in PyTorch — they are already cheap.

3 Build process — six steps, in order



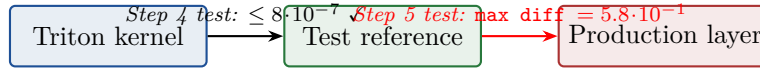
Step 1 — Catmull-Rom kernel. Port `_catmull_rom_eval` (the leaf operation) first. It is the simplest unit and validates the basic Triton plumbing. *Acceptance:* forward parity $\leq 10^{-4}$ vs reference at 6 shapes; speedup $\geq 5\times$ at canonical shape.

Step 2 — Fused inner (no-skip). Fuse the gather + per-sign CR + σ -mask + mean reduce into one kernel. This is where the real speedup lives — it eliminates the $(T, k, 2, d)$ intermediate that the PyTorch path materialises. *Acceptance:* forward parity $\leq 10^{-4}$; speedup $\geq 10\times$ at canonical shape; arity-agnostic over $k = 2 \dots 6$.

Step 3 — Sensitivity sweep. Map the kernel’s performance landscape across $(T, d, k, G, \text{BLOCK_T}, \text{BLOCK_D})$. Don’t skip this: block-size defaults can cost $2\times$ speedup. We found `BLOCK_T = 16` was $2\times$ faster than the default 32 at $d = 4$ — pure default-tuning win.

Step 4 — Skip-variant extension. Most production paths have an inner skip (residual / highway). Inline a d -by- d matvec for the gate logit: $\ell_c = \sum_{e=0}^{d-1} W_{e,c} \cdot h_e + b_c$, costing $O(d^2)$ extra work per (t, k, d_c) but the gather + CR remain dominant for $d \leq 32$.

Step 5 — Semantic alignment with production. (Critical.) This is the step that catches bugs before they regress live training. Run a parity test against the *actual production layer*, not against your own test reference (which may differ from production).



In our case, the kernel applied $\tanh(\text{CR}(x))$ and used a clamped- h_v residual. Production `SignedKANLayer._forward` did *neither* — raw CR output, raw h_v residual. The fix:

- Drop `tl.extra.libdevice.tanh(cr)` → just `cr` in both kernels.
- Replace `x_clamp` → `x` in the highway residual term.
- Update the PyTorch reference in tests + sensitivity + autograd backward to match.

After fix: parity vs production layer = $1.2 \cdot 10^{-7}$ highway, $9.9 \cdot 10^{-8}$ no-skip.

Step 6 — Autograd wrap. The kernel returns a tensor without an autograd graph. Wrap each variant in a `torch.autograd.Function`. Forward calls the Triton kernel; backward (v0) re-runs the forward through the PyTorch reference under autograd. Slower than a fused backward but correct without writing one. *Acceptance:* $\partial x, \partial \text{coef}_{\pm}, \partial W, \partial b$ all match reference to $\leq 10^{-4}$.

Step 7 — Live integration with env-gating. Add a fast-path in `_forward_impl` that dispatches to the autograd-wrapped kernel when conditions match. **Gate the dispatch behind an env var, default OFF**, until the backward kernel matches the forward path's memory profile.

```

def _can_use_triton_inner(self, x):
    if int(os.environ.get("HSIKAN_TRITON_KERNEL", "0")) == 0:
        return False
    if not x.is_cuda: return False
    if self.n_branches != 2: return False
    if self.inner_skip not in ("none", "highway"): return False
    if not isinstance(self.inner, BatchedCatmullRomActivation):
        return False
    try: import triton # noqa
    except ImportError: return False
    return True
  
```

4 Test tiers

Tier 0: Build sanity — kernel compiles, runs once

Tier 1: Forward parity vs explicit PyTorch ref

Tier 2: Production-layer parity vs the actual nn.Module

Tier 3: Autograd parity — $\partial x, \partial \text{coef}, \partial W, \partial b$

Tier 4: Performance — speedup $\geq$ threshold

Tier 5: Integration — install/uninstall idempotent

Tier 6: End-to-end — BA training kernel-OFF + kernel-ON

tier	what passes
0	kernel returns expected-shape tensor; no compile error
1	$ \text{out}_{\text{kernel}} - \text{out}_{\text{ref}} _{\max} < 10^{-4}$ at ≥ 3 shapes
2	parity vs the production SignedKANLayer.forward (or whatever the production caller is) $\leq 10^{-4}$
3	gradient parity $\leq 10^{-4}$ for every differentiable input
4	speedup $\geq 5\times$ (CR), $\geq 10\times$ (fused inner) at canonical shape
5	install + uninstall is a semantic no-op; idempotent under repeated calls
6	BA training with HSIKAN_TRITON_KERNEL=0/1 produces comparable AUC ($\pm 1\%$); kernel-OFF run does not OOM

The current suite has 31 tests covering tiers 1, 3, 4, 5, 6. **Tier 2 was missing in the first pass** — that’s how the semantic divergence shipped to integration before being caught.

5 The math — forward and backward in closed form

This section gives the kernel’s exact forward function and the closed-form gradients the fused backward kernel needs to compute.

Notation. V = number of vertices, T = number of cycles, k = arity, d = hidden dim, G = grid (control-point count), $S = 2$ = sign branches. $x \in \mathbb{R}^{V \times d}$ (vertex embeddings), $v \in \{0, \dots, V-1\}^{T \times k}$ (**triad_v**), $\sigma \in \{+1, -1\}^{T \times k}$ (**triad_sigma**), $C^+, C^- \in \mathbb{R}^{d \times G}$ (**coef_pos**, **coef_neg**), $W \in \mathbb{R}^{d \times d}$, $b \in \mathbb{R}^d$ (**gate_w**, **gate_b**), $\text{agg} \in \mathbb{R}^{T \times S \times d}$ (kernel output).

5.1 Forward (Highway variant, matches production)

For each cycle $t \in [T]$, slot $i \in [k]$, channel $c \in [d]$:

$$h_{t,i,c} = x_{v_{t,i},c}, \quad \hat{h}_{t,i,c} = \text{clamp}(h_{t,i,c}, -1, 1).$$

Catmull-Rom evaluation. Let $u = (\hat{h} + 1) \cdot \frac{G-1}{2}$, $j = \lfloor u \rfloor$, $\tau = u - j$. The four blending weights are

$$\begin{aligned} w_{-1}(\tau) &= \frac{1}{2}(-\tau^3 + 2\tau^2 - \tau), & w_0(\tau) &= \frac{1}{2}(3\tau^3 - 5\tau^2 + 2), \\ w_{+1}(\tau) &= \frac{1}{2}(-3\tau^3 + 4\tau^2 + \tau), & w_{+2}(\tau) &= \frac{1}{2}(\tau^3 - \tau^2), \end{aligned}$$

so for sign $s \in \{+, -\}$:

$$\text{CR}_{t,i,c}^s = \sum_{p \in \{-1, 0, +1, +2\}} w_p(\tau_{t,i,c}) \cdot C_{c, \text{clip}(j_{t,i,c}+p, 0, G-1)}^s.$$

Highway gate. Per- (t, i, c) sigmoid of a d -by- d matvec:

$$\ell_{t,i,c} = \sum_{e=0}^{d-1} W_{e,c} \cdot h_{t,i,e} + b_c, \quad T_{t,i,c} = \sigma(\ell_{t,i,c}) = \frac{1}{1 + e^{-\ell_{t,i,c}}}.$$

Highway mix. Raw h residual — production semantics, no **tanh** wrap:

$$H_{t,i,c}^s = T_{t,i,c} \cdot \text{CR}_{t,i,c}^s + (1 - T_{t,i,c}) \cdot h_{t,i,c}.$$

σ -masked mean.

$$m_{t,i}^s = \mathbb{I}[\sigma_{t,i} = s], \quad n_t^s = \max\left(1, \sum_i m_{t,i}^s\right),$$

$$\boxed{\text{agg}_{t,c}^s = \frac{1}{n_t^s} \sum_{i=0}^{k-1} m_{t,i}^s \cdot H_{t,i,c}^s.}$$

The no-skip variant drops the gate: $H_{t,i,c}^s \mapsto \text{CR}_{t,i,c}^s$.

5.2 Backward — what the fused backward kernel must compute

Given the upstream gradient $g_{t,c}^s = \partial\mathcal{L}/\partial\text{agg}_{t,c}^s$, the backward outputs five gradients: $\partial\mathcal{L}/\partial x$, $\partial\mathcal{L}/\partial C^+$, $\partial\mathcal{L}/\partial C^-$, $\partial\mathcal{L}/\partial W$, $\partial\mathcal{L}/\partial b$.

Per-cycle-slot pre-gradient (broadcast over masked slots, divide by count):

$$\tilde{g}_{t,i,c}^s = \frac{m_{t,i}^s}{n_t^s} \cdot g_{t,c}^s.$$

This is the only $(T \times k \times S \times d)$ tensor the backward needs — and it can be streamed inside the kernel without materialising it.

Highway-gate locals (pre-computed once per (t, i, c)):

$$\Delta_{t,i,c}^s = \text{CR}_{t,i,c}^s - h_{t,i,c}, \quad \partial T / \partial \ell = T(1 - T).$$

5.2.1 (a) $\partial\mathcal{L}/\partial x$ — three contributions

Three additive paths from agg back to x :

(i) **Residual path**

$$\left. \frac{\partial \text{agg}_{t,c}^s}{\partial h_{t,i,c}} \right|_{\text{res}} = \frac{m_{t,i}^s}{n_t^s} (1 - T_{t,i,c}).$$

(ii) **Gate-logit path**

(the gate uses *all* d components of $h_{t,i}$, so $\partial \text{agg}_c / \partial h_e \neq 0$ for $e \neq c$): $\left. \frac{\partial \text{agg}_{t,c}^s}{\partial h_{t,i,e}} \right|_{\text{gate}} = \frac{m_{t,i}^s}{n_t^s} \cdot \Delta_{t,i,c}^s \cdot T_{t,i,c} (1 - T_{t,i,c}) \cdot W_{e,c}.$

(iii) **Spline-input path**

$$\left. \frac{\partial \text{agg}_{t,c}^s}{\partial h_{t,i,c}} \right|_{\text{CR}} = \frac{m_{t,i}^s}{n_t^s} \cdot T_{t,i,c} \cdot \frac{\partial \text{CR}_{t,i,c}^s}{\partial h_{t,i,c}},$$

with $\partial \text{CR}_{t,i,c}^s / \partial h_{t,i,c}$ from the CR closed form (zero on **clamp**-saturated inputs):

$$\frac{\partial \text{CR}_{t,i,c}^s}{\partial h_{t,i,c}} = \mathbb{K}[|h_{t,i,c}| < 1] \cdot \frac{G-1}{2} \sum_p w'_p(\tau_{t,i,c}) \cdot C_{c,\text{clip}(j+p)}^s,$$

where $w'_{-1}(\tau) = \frac{1}{2}(-3\tau^2 + 4\tau - 1)$, $w'_0(\tau) = \frac{1}{2}(9\tau^2 - 10\tau)$, $w'_{+1}(\tau) = \frac{1}{2}(-9\tau^2 + 8\tau + 1)$, $w'_{+2}(\tau) = \frac{1}{2}(3\tau^2 - 2\tau)$.

The kernel scatter-adds the three paths into $\partial\mathcal{L}/\partial x$ at vertex index $v_{t,i}$:

$$\frac{\partial \mathcal{L}}{\partial x_{v,e}} = \sum_{(t,i): v_{t,i}=v} \sum_s \tilde{g}_{t,i,c}^s \cdot [(1-T)_{t,i,e} \delta_{e,c} + \Delta_{t,i,c}^s T(1-T)_{t,i,c} W_{e,c} + T_{t,i,e} \partial \text{CR} / \partial h_{t,i,e} \delta_{e,c}].$$

5.2.2 (b) $\partial\mathcal{L}/\partial C^+$, $\partial\mathcal{L}/\partial C^-$ — control-point gradients

The CR output is *linear* in the control points, so

$$\frac{\partial \text{CR}_{t,i,c}^s}{\partial C_{c,q}^s} = \sum_{p \in \{-1,0,+1,+2\}} w_p(\tau_{t,i,c}) \cdot \mathbb{K}[\text{clip}(j_{t,i,c} + p, 0, G-1) = q].$$

Scatter-add over (t, i) :

$$\boxed{\frac{\partial \mathcal{L}}{\partial C_{c,q}^s} = \sum_{t,i} \tilde{g}_{t,i,c}^s \cdot T_{t,i,c} \cdot \sum_p w_p(\tau_{t,i,c}) \cdot \mathbb{K}[\text{clip}(j + p) = q].}$$

Only sign- s slots contribute (via $m_{t,i}^s$ in $\tilde{g}^s$).

5.2.3 (c) $\partial\mathcal{L}/\partial W, \partial\mathcal{L}/\partial b$ — gate Linear gradients

$$\frac{\partial\mathcal{L}}{\partial W_{e,c}} = \sum_{s,t,i} \tilde{g}_{t,i,c}^s \cdot \Delta_{t,i,c}^s \cdot T_{t,i,c}(1 - T_{t,i,c}) \cdot h_{t,i,e},$$

$$\frac{\partial\mathcal{L}}{\partial b_c} = \sum_{s,t,i} \tilde{g}_{t,i,c}^s \cdot \Delta_{t,i,c}^s \cdot T_{t,i,c}(1 - T_{t,i,c}).$$

Both are reductions over (s, t, i) — no scatter, just an accumulator-per- (e, c) or per- (c) in registers, atomic-add-flushed at block end.

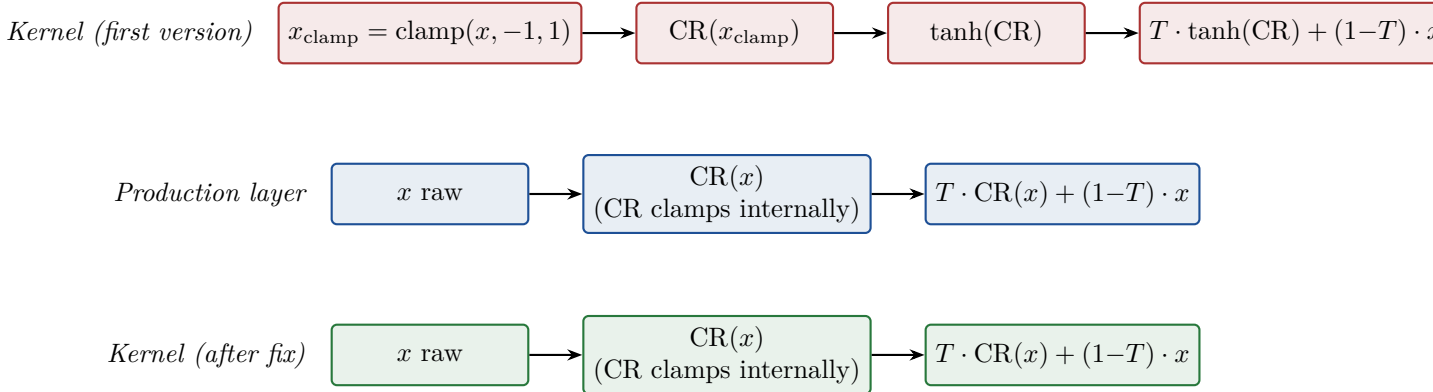
5.3 Memory profile of the fused backward

The PyTorch-ref backward (current v0) materialises the (T, k, S, d) post-highway intermediate inside the autograd graph. At Slashdot scale ($T = 200K, k = 4, S = 2, d = 4$) that is ~ 25 MB per layer; across multiple layers + autograd-retained inputs the peak hits ~ 6 GiB on BA-scale runs.

The fused backward kernel’s working set per program block is $O(\text{BLOCK_T} \cdot d \cdot k)$ — under 1 MB at canonical block sizes. The only $O(T \cdot k \cdot d)$ allocation is `agg_grad_in` (the upstream g , just the kernel input), and the only $O(T \cdot d)$ write is `x_grad_out` via scatter-add.

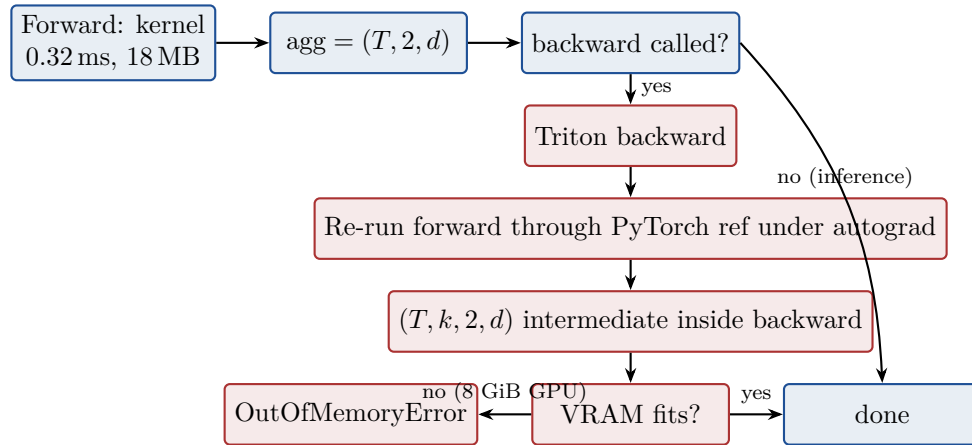
Predicted memory for the fused backward at Slashdot SOTA shape: ~ 30 MB peak (vs 6 GiB for the v0 ref). That is the gap that has to close before live integration can be enabled at training time on 8 GiB GPUs.

6 Forward-semantics comparison: before vs after the fix



The kernel’s first version applied two semantic transforms that production never did. Both came from a misread of the original PyTorch path. The fix is two characters of Triton (drop `tanh`) and one rename ($x_{\text{clamp}} \rightarrow x$) — but you have to *see* the divergence first, which requires Tier-2 testing.

7 The OOM trap and why integration is env-gated



The Triton autograd **forward** is fast and memory-light — but the **backward is a PyTorch re-compute**. It allocates the very intermediate the kernel was supposed to avoid. On the BA end-to-end test ($T \sim 50K$, $k = 4$, $d = 8$, multiple layers), that intermediate hits 6 GiB and OOMs an 8 GiB GPU.

This is why live integration is env-gated, default OFF. The kernel is ready for forward-only / inference paths today; training-time enabling requires the fused backward kernel.

8 Common pitfalls — checklist

- ❑ **Step 5 / Tier 2 — production parity, not test-ref parity.** The most expensive bug we shipped. Always include a parity test against the actual nn.Module the kernel is replacing.
- ❑ **BLOCK_T default.** Start at the next power of 2 above your typical batch dimension and sweep down to 8. We found 16 was $2\times$ faster than 32 at $d = 4$.
- ❑ **CR domain clamping.** The CR spline only supports $x \in [-1, 1]$. Production `_catmull_rom_eval` clamps internally; if your kernel does too, make sure the residual term uses raw x , not the clamped one.
- ❑ **Backward memory.** A correct-but-slow PyTorch-ref backward can OOM at training scale even when the forward is fine. Plan the backward kernel before claiming integration is done.
- ❑ **Autograd wrapping.** A bare Triton call returns a tensor without a graph — `loss.backward()` will silently miss it. Wrap in `torch.autograd.Function`.
- ❑ **Env-gate the integration** until the full forward+backward path is safe at production scale. Default OFF.
- ❑ **Speedup numbers are PyTorch-ref-relative.** When you change the reference (e.g. drop `tanh`), the speedup number changes too — not because the kernel got slower but because the baseline got faster.

9 Files of record

file	role
<code>signedkan_wip/src/triton_kernels.py</code>	kernels, autograd wrappers, install/uninstall
<code>signedkan_wip/src/triton_kernels_sensitivity.py</code>	parameter sensitivity sweep
<code>signedkan_wip/tests/test_triton_kernels.py</code>	py tests, tiers 1/3/4/5/6
<code>signedkan_wip/src/signedkan.py</code>	live integration fast-path
<code>docs/triton_kernel_integration_tutorial_2026-05-09.md</code>	(markdown)
<code>reports/triton_kernel_integration_tutorial_2026-05-09(LaTeX)</code>	(LaTeX)

10 Quick reference — running the suite

```
# Full suite (<= 60s)
python -m pytest signedkan_wip/tests/test_triton_kernels.py -v

# Skip the slow end-to-end BA training test (<= 5s)
python -m pytest signedkan_wip/tests/test_triton_kernels.py -v -k "not end_to_end_ba"

# Sensitivity sweep (~2 min)
python -m signedkan_wip.src.triton_kernels_sensitivity

# Try training with the kernel ON
# (forward-only OK; backward may OOM on small GPUs until fused-backward lands)
HSIKAN_TRITON_KERNEL=1 python -m signedkan_wip.src.run_final_cell --dataset
    bitcoin_alpha
```